什么是生命周期?

每个 Vue 实例在被创建时都要经过一系列的初始化过程，例如，需要设置数据监听、编译模板、将实例挂载到 DOM 并在数据变化时更新 DOM 等。同时在这个过程中也会运行一些叫做生命周期钩子的函数，这给了用户在不同阶段添加自己的代码的机会。

函数

beforecreate

在实例初始化之后，此时的数据观察和事件机制都未形成，不能获得 DOM 节点。

created

实例已经创建完成之后被调用。在这一步，实例已完成以下的配置：数据观测（data observer），属性和方法的运算， watch/event 事件回调。然而，挂载阶段还没开始。

beforemount

在挂载开始之前被调用：这个过程是在模版已经在内存中编译完成， render 函数首次被调用，此时完成了虚拟 DOM 的构建，但并未被渲染。

mounted

这个过程在模版挂载之后被调用，页面完成渲染，所以在这之后，我们可以操作和访问 DOM 元素。

beforeupdate

当数据更新时调用，在这一阶段 DOM 会和更改过的内容同步。

updated

由于数据更改导致的虚拟 DOM 重新渲染和打补丁，在这之后会调用该钩子。当这个钩子被调用时，组件 DOM 已经更新，所以你现在可以执行依赖于 DOM 的操作。然而在大多数情况下，你应该避免在此期间更改状态，因为这可能会导致更新无限循环。

beforeDestroy

实例销毁之前调用。在这一步，实例仍然完全可用。

destroyed

Vue 实例销毁后调用。调用后，Vue 实例指示的所有东西都会解绑定，所有的事件监听器会被移除，所有的子实例也会被销毁。

```
<div>
    <h3>这是生命周期函数演示</h3>
    <h4 ref="m">{{msg}}</h4>
    <button @click="setMsg()">修改</button>
</div>


export default{
    data(){
        return{
            msg:"生命周期钩子函数"
        }
    },
    methods:{
        setMsg(){
            this.msg="修改后"
            // console.log("被调用了")
        }
    },
    // 组件挂载前
    beforeCreate() {
        // 表示，实例完全被创建之前会执行的函数，data和methods的数据并没有初始化
        // console.log("--beforeCreate--")
        // console.log(this.msg);
        // this.setMsg()
    },
    created() {//创建后
        // data和methods已经被初始化好了，如果要调用data中的属性或methods中的方法，可以在created中调用
        // 但是不能获取到dom元素
        // console.log("--created--")
        // console.log(this.msg);
        // console.log(this.$refs.m);
        // this.setMsg()
    },
    beforeMount() {//挂载前
      // 模板已经在内存中编译好了，但是尚未挂载到页面中去，此时页面还是旧的
        // console.log("--beforeMount--")
        // console.log(this.msg);
        // console.log(this.$refs.m);
    },
    mounted() {//挂载后
        // 组件已经脱离了创建阶段，进入到了运行阶段
```

```
        // 如果需要操作dom节点，最早要在mounted进行
        // console.log("--mounted--")
        // console.log(this.msg);
        // console.log(this.$refs.m);
    },
    beforeUpdate() {//修改前
    // 当beforeUpdate执行的时候，页面的数据还是旧的，data中的数据是最新的，页面没有和最新的数据保持相同
        // console.log("--beforeUpdate--")
        // console.log("data的数据：",this.msg);
        // console.log("页面的数据：",this.$refs.m.innerText);
    },
    updated() {//修改后
        // 数据和页面已经保持了同步，都是最新的
        // console.log("--updated--")
        // console.log("data的数据：",this.msg);
        // console.log("页面的数据：",this.$refs.m.innerText);
    },
    beforeDestroy() {//销毁前
    // 页面销毁的时候要保存一些数据，就可以监听这个销毁的生命周期函数
        console.log("--beforeDestroy--")
        console.log("data的数据：",this.msg);
        console.log("页面的数据：",this.$refs.m.innerText);
    },
    destroyed() {//销毁后
    // 组件已经完全被销毁，此时所有的数据和方法、指令等等，都不可以用了
        console.log("--destroyed--")
        console.log("data的数据：",this.msg);
        console.log("页面的数据：",this.$refs.m.innerText);
    }


}
```

事件车

（一般项目不大就利用,事件车，公共的Vue实例进行传递数据，

利用$emit发射自定义事件 和 $on监听自定义事件）

默认创建并注册好了三个组件 AA Home Product

1.创建eventBus.js

```
// 建立的公共js文件(事件车),主要用来传递信息
// 引入vue.js
import Vue from "vue"
// 创建空的Vue实例
```

```
var event =new Vue();
export default event;
```

2. 在Vue.vue中引入

```
// 在需要传递数据的地方(组件)引入
import bus from "./eventBus.js"
```

3. 定义一个方法利用$emit发送数据

```
//在Product.vue中
<button @click="fn">点击发射</button>

export default{
    methods:{
            fn(){
              bus.$emit("msg","这是商品页传递过来的数据")//也可以发送data中的数据出去
            }
    }
}
```

4. 利用$on监听自定义事件

```
//在Home.vue中
//接收到后就可以使用它
<h2>{{mymsg}}</h2>

export default{
        mounted() { //组件挂载时触发的钩子函数
            // 接收信息
            bus.$on("msg", val =>{
                this.mymsg = val
            } )
        },
        data(){
            return{
                mymsg:"" //如果传来的是数组或者对象，就在这里定义成null类型接收
            }
        }
}
```